

LAPORAN PRAKTIKUM PEMROGRAMAN WEB

MODUL 9



**Universitas
Telkom**

Disusun Oleh:

Nama: Davin Aileenskie Sofiandy

NIM: 707012500135

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

A. Tujuan

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep HMVC pada CodeIgniter 3.
2. Mengimplementasikan struktur modular (feature-based).
3. Membuat REST API berbasis JSON.
4. Mengintegrasikan Model dengan REST Controller.
5. Menggunakan AJAX untuk komunikasi asynchronous.
6. Membuat fitur CRUD tanpa refresh halaman.
7. Menguji API menggunakan Postman.
8. Mengintegrasikan frontend dengan REST API

B. Alat dan Bahan

1. XAMPP (Apache & MySQL aktif)
 2. PHP 7.x
 3. CodeIgniter 3
 4. Library HMVC (Modular Extensions – MX)
 5. Library REST Server (chriskacerguis)
 6. Postman
 7. VS Code / Code Editor
 8. Browser (Chrome/Firefox)
7. Apa perbedaan request synchronous dan asynchronous?

C. Langkah-Langkah Praktikum

1. Menyiapkan project CodeIgniter 3 pada folder Pekan-09/ci3_project.
2. Menjalankan Apache dan MySQL melalui XAMPP.
3. Membuat database webpro di phpMyAdmin.
4. Mengimport file database_pekan09.sql untuk membuat tabel users dan posts.
5. Membuat struktur HMVC pada folder application/modules.
6. Membuat modul Crudjs untuk fitur CRUD AJAX.
7. Membuat controller, model, dan view pada modul Crudjs.
8. Membuat folder uploads/posts untuk menyimpan gambar post.
9. Membuat library Jwt.php untuk proses token login.
10. Membuat modul Api untuk REST API.
11. Membuat controller Auth.php untuk register, login, logout, dan detail akun.
12. Membuat controller Post.php untuk mengelola data post melalui API.
13. Membuat model User_model.php dan Post_model.php.
14. Mengatur route API pada file application/config/routes.php.
15. Menguji register dan login API menggunakan Postman.
16. Mengecek data register pada tabel users di phpMyAdmin.
17. Menguji endpoint post menggunakan Bearer Token dari hasil login.

18. Membuka CRUD AJAX di browser untuk memastikan data dapat diproses tanpa refresh halaman.
19. Mendokumentasikan hasil praktikum dengan screenshot browser, Postman, dan phpMyAdmin.

D. Hasil dan Dokumentasi Program

```
<?php
// Mencegah file controller diakses langsung tanpa lewat CodeIgniter.
defined('BASEPATH') OR exit('No direct script access allowed');

// Controller API untuk fitur autentikasi user seperti register, login, logout, dan data user login.
class Auth extends MX_Controller {

    // Constructor dijalankan otomatis saat controller dipanggil.
    public function __construct()
    {
        // Memanggil constructor bawaan MX_Controller.
        parent::__construct();
        // Memuat model User_model dari module Api dengan alias User_model.
        $this->load->model('Api/User_model', 'User_model');
        // Memuat library JWT untuk membuat dan memverifikasi token.
        $this->load->library('jwt');
        // Memuat library form_validation untuk kebutuhan validasi input.
        $this->load->library('form_validation');
    }

    // Endpoint API untuk mendaftarkan user baru.
    public function register()
    {
        // Mengatur response menjadi JSON.
        $this->_json();

        // Menjalankan proses register dalam blok try agar error bisa ditangkap.
        try {
            // Mengambil input request dari JSON body atau form POST.
            $input = $this->_request_input();

            // Mengecek apakah field name, email, dan password kosong.
            if (empty($input['name']) || empty($input['email']) || empty($input['password'])) {
                // Mengirim response error jika input wajib belum lengkap.
                return $this->_response(false, 'Name, email, and password are required', null, 400);
            }

            // Mengecek apakah format email valid.
            if (!filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
                // Mengirim response error jika format email salah.
                return $this->_response(false, 'Invalid email format', null, 400);
            }
        }
    }
}
```

```

        // Mengecek apakah email sudah terdaftar.
        if ($this->User_model->email_exists($input['email'])) {
            // Mengirim response conflict jika email sudah digunakan.
            return $this->_response(false, 'Email already registered', null,
409);
        }

        // Mengecek apakah panjang password minimal 6 karakter.
        if (strlen($input['password']) < 6) {
            // Mengirim response error jika password terlalu pendek.
            return $this->_response(false, 'Password must be at least 6
characters', null, 400);
        }

        // Menyiapkan data user yang akan disimpan ke database.
        $user_data = array(
            // Membersihkan nama dari karakter HTML.
            'name' => htmlspecialchars($input['name']),
            // Mengubah email menjadi huruf kecil agar konsisten.
            'email' => strtolower($input['email']),
            // Mengenkripsi password menggunakan bcrypt sebelum disimpan.
            'password' => password_hash($input['password'], PASSWORD_BCRYPT)
        );

        // Menyimpan user baru melalui model dan mengambil ID user.
        $user_id = $this->User_model->create($user_data);
        // Mengecek apakah proses pembuatan user gagal.
        if (!$user_id) {
            // Mengirim response error server jika user gagal dibuat.
            return $this->_response(false, 'Failed to create user', null, 500);
        }

        // Mengambil kembali data user yang baru dibuat.
        $user = $this->User_model->get_by_id($user_id);
        // Membuat token JWT berisi data dasar user.
        $token = $this->jwt->create(array(
            // Menyimpan ID user ke payload token.
            'id' => $user->id,
            // Menyimpan nama user ke payload token.
            'name' => $user->name,
            // Menyimpan email user ke payload token.
            'email' => $user->email
        ));

        // Mengirim response sukses register beserta data user dan token.
        return $this->_response(true, 'User registered successfully', array(
            // ID user yang baru terdaftar.
            'id' => $user->id,
            // Nama user yang baru terdaftar.
            'name' => $user->name,
            // Email user yang baru terdaftar.
            'email' => $user->email,
            // Waktu user dibuat.

```

```

        'created_at' => $user->created_at,
        // Token akses untuk autentikasi request berikutnya.
        'access_token' => $token,
        // Jenis token yang digunakan pada header Authorization.
        'token_type' => 'Bearer'
    ), 201);
    // Menangkap exception jika terjadi error server.
} catch (Exception $e) {
    // Mengirim response error server beserta pesan exception.
    return $this->_response(false, 'Server error: ' . $e->getMessage(),
null, 500);
}
}

// Endpoint API untuk login user.
public function login()
{
    // Mengatur response menjadi JSON.
    $this->_json();

    // Menjalankan proses login dalam blok try.
    try {
        // Mengambil input request dari JSON body atau form POST.
        $input = $this->_request_input();

        // Mengecek apakah email dan password sudah dikirim.
        if (empty($input['email']) || empty($input['password'])) {
            // Mengirim response error jika email/password kosong.
            return $this->_response(false, 'Email and password are required',
null, 400);
        }

        // Mengambil data user berdasarkan email.
        $user = $this->User_model->get_by_email($input['email']);
        // Mengecek apakah user tidak ditemukan atau password salah.
        if (!$user || !password_verify($input['password'], $user->password)) {
            // Mengirim response unauthorized jika kredensial salah.
            return $this->_response(false, 'Invalid email or password', null,
401);
        }

        // Membuat token JWT untuk user yang berhasil login.
        $token = $this->jwt->create(array(
            // Menyimpan ID user ke payload token.
            'id' => $user->id,
            // Menyimpan nama user ke payload token.
            'name' => $user->name,
            // Menyimpan email user ke payload token.
            'email' => $user->email
        ));

        // Mengirim response sukses login beserta token.
        return $this->_response(true, 'Login successful', array(
            // ID user yang login.

```

```

        'id' => $user->id,
        // Nama user yang login.
        'name' => $user->name,
        // Email user yang login.
        'email' => $user->email,
        // Token akses untuk autentikasi API.
        'access_token' => $token,
        // Jenis token untuk header Authorization.
        'token_type' => 'Bearer'
    ));
    // Menangkap exception jika terjadi error server.
} catch (Exception $e) {
    // Mengirim response error server.
    return $this->_response(false, 'Server error: ' . $e->getMessage(),
null, 500);
}

}

// Endpoint API logout.
public function logout()
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengirim response logout berhasil, token biasanya dihapus di sisi
client.
    return $this->_response(true, 'Logout successful');
}

// Endpoint API untuk mengambil data user yang sedang login.
public function me()
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Memverifikasi token dan mengambil payload user.
    $payload = $this->_auth_payload();
    // Jika token tidak valid, proses dihentikan karena response error sudah
dikirim.
    if (!$payload) {
        return;
    }

    // Mengambil data user berdasarkan ID dari payload token.
    $user = $this->User_model->get_by_id($payload->id);
    // Mengecek apakah user masih ada di database.
    if (!$user) {
        // Mengirim response error jika user tidak ditemukan.
        return $this->_response(false, 'User not found', null, 404);
    }

    // Mengirim response sukses berisi detail user.
    return $this->_response(true, 'User detail', array(
        // ID user.
        'id' => $user->id,
        // Nama user.

```

```

        'name' => $user->name,
        // Email user.
        'email' => $user->email,
        // Waktu user dibuat.
        'created_at' => $user->created_at
    ));
}

// Method private untuk mengambil dan memverifikasi token JWT dari request.
private function _auth_payload()
{
    // Mengambil token dari header Authorization request.
    $token = $this->jwt->get_token_from_request();
    // Memverifikasi token dan mengambil payload-nya.
    $payload = $this->jwt->verify($token);
    // Mengecek apakah token tidak valid atau kosong.
    if (!$payload) {
        // Mengirim response unauthorized jika token gagal diverifikasi.
        $this->response(false, 'Unauthorized or invalid token', null, 401);
        // Mengembalikan false sebagai penanda autentikasi gagal.
        return false;
    }
    // Mengembalikan payload token jika valid.
    return $payload;
}

// Method private untuk mengatur response sebagai JSON.
private function _json()
{
    // Bersihkan buffer hanya jika memang ada buffer aktif agar tidak memicu
notice.
    if (ob_get_level() > 0) {
        ob_clean();
    }

    // Gunakan output class CI supaya header JSON lebih konsisten.
    $this->output->set_content_type('application/json', 'utf-8');
}

// Method private untuk mengambil input request.
private function _request_input()
{
    // Ambil data mentah request untuk mendukung JSON dan x-www-form-
urlencoded.
    $raw_input = trim(file_get_contents('php://input'));

    // Prioritaskan body JSON jika memang valid.
    if ($raw_input !== '') {
        $decoded = json_decode($raw_input, true);
        if (json_last_error() === JSON_ERROR_NONE && is_array($decoded)) {
            return $decoded;
        }
    }
}

```

```

        // Untuk POST, CodeIgniter bisa langsung membaca form-data atau x-www-form-
        urlencoded.
        $post_input = $this->input->post(NULL, true);
        if (is_array($post_input) && !empty($post_input)) {
            return $post_input;
        }

        // Untuk PUT/PATCH/DELETE, coba baca input stream dari CI.
        $stream_input = $this->input->input_stream(NULL, true);
        if (is_array($stream_input) && !empty($stream_input)) {
            return $stream_input;
        }

        // Fallback terakhir untuk raw body query-string style dari Postman.
        if ($raw_input !== '') {
            parse_str($raw_input, $parsed_input);
            if (is_array($parsed_input) && !empty($parsed_input)) {
                return $parsed_input;
            }
        }

        // Jika semua cara gagal, kembalikan array kosong.
        return array();
    }

    // Method private untuk mengirim response JSON standar.
    private function _response($status, $message, $data = null, $code = 200)
    {
        // Mengatur HTTP status code response.
        http_response_code($code);
        // Membuat struktur dasar response.
        $response = array('status' => $status, 'message' => $message);
        // Mengecek apakah ada data tambahan yang perlu dikirim.
        if ($data !== null) {
            // Menambahkan data ke response.
            $response['data'] = $data;
        }
        // Mencetak response dalam format JSON.
        echo json_encode($response, JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE);
        // Menghentikan eksekusi agar tidak ada output tambahan.
        exit;
    }
}

<?php
// Mencegah file controller diakses langsung tanpa lewat CodeIgniter.
defined('BASEPATH') OR exit('No direct script access allowed');

// Controller API untuk mengelola data post dengan method GET, POST, PUT, dan
DELETE.
class Post extends MX_Controller {

    // Constructor dijalankan otomatis saat controller dipanggil.

```



```

public function __construct()
{
    // Memanggil constructor bawaan MX_Controller.
    parent::__construct();
    // Memuat model Post_model untuk akses database post.
    $this->load->model('Post_model');
    // Memuat library JWT untuk validasi token API.
    $this->load->library('jwt');
}

// Method utama untuk mengarahkan request API berdasarkan HTTP method dan ID.
public function handle($id = null)
{
    // Mengambil HTTP method request lalu mengubahnya menjadi huruf besar.
    $method = strtoupper($this->input->method(TRUE));

    // Jika method GET tanpa ID, arahkan ke index untuk mengambil semua post.
    if ($method === 'GET' && $id === null) {
        // Memanggil method index.
        return $this->index();
    }
    // Jika method GET dengan ID, arahkan ke show untuk mengambil satu post.
    if ($method === 'GET' && $id !== null) {
        // Memanggil method show berdasarkan ID.
        return $this->show($id);
    }
    // Jika method POST, arahkan ke create untuk membuat post baru.
    if ($method === 'POST') {
        // Memanggil method create.
        return $this->create();
    }
    // Jika method PUT dengan ID, arahkan ke update untuk mengubah post.
    if ($method === 'PUT' && $id !== null) {
        // Memanggil method update berdasarkan ID.
        return $this->update($id);
    }
    // Jika method DELETE dengan ID, arahkan ke delete untuk menghapus post.
    if ($method === 'DELETE' && $id !== null) {
        // Memanggil method delete berdasarkan ID.
        return $this->delete($id);
    }

    // Mengirim response jika kombinasi method dan URL tidak diizinkan.
    return $this->_response(false, 'Method not allowed', null, 405);
}

// Endpoint untuk mengambil semua data post.
public function index()
{
    // Mengatur response menjadi JSON.
    $this->_json();
    // Mengecek token JWT sebelum mengakses data.
    if (!$this->_auth_payload()) return;
}

```

```

        // Mengambil semua data post dari model.
        $posts = $this->Post_model->get_all();
        // Mengirim response sukses berisi semua post.
        return $this->_response(true, 'Posts retrieved successfully', $posts);
    }

    // Endpoint untuk mengambil satu data post berdasarkan ID.
    public function show($id)
    {
        // Mengatur response menjadi JSON.
        $this->_json();
        // Mengecek token JWT sebelum mengakses data.
        if (!$this->_auth_payload()) return;

        // Mengambil data post berdasarkan ID.
        $post = $this->Post_model->get_by_id($id);
        // Mengecek apakah post tidak ditemukan.
        if (!$post) {
            // Mengirim response error jika post tidak ada.
            return $this->_response(false, 'Post not found', null, 404);
        }

        // Mengirim response sukses berisi detail post.
        return $this->_response(true, 'Post retrieved successfully', $post);
    }

    // Endpoint untuk membuat data post baru.
    public function create()
    {
        // Mengatur response menjadi JSON.
        $this->_json();
        // Mengecek token JWT sebelum membuat data.
        if (!$this->_auth_payload()) return;

        // Ambil input request dengan dukungan JSON dan form-data dari Postman.
        $input = $this->_request_input();
        // Mengecek apakah field wajib belum lengkap.
        if (empty($input['title']) || empty($input['author']) ||
empty($input['article'])) {
            // Mengirim response error validasi.
            return $this->_response(false, 'Title, author, and article are
required', null, 400);
        }

        // Menyimpan post baru ke database melalui model.
        $post_id = $this->Post_model->create(array(
            // Mengisi judul post dari input JSON.
            'title' => $input['title'],
            // Mengisi nama author dari input JSON.
            'author' => $input['author'],
            // Mengisi artikel dari input JSON.
            'article' => $input['article']
        ));
    }

```

```

        // Mengirim response sukses beserta data post yang baru dibuat.
        return $this->_response(true, 'Post created successfully', $this->Post_model->get_by_id($post_id), 201);
    }

    // Endpoint untuk mengubah data post berdasarkan ID.
    public function update($id)
    {
        // Mengatur response menjadi JSON.
        $this->_json();
        // Mengecek token JWT sebelum mengubah data.
        if (!$this->_auth_payload()) return;

        // Mengecek apakah post dengan ID tersebut ada.
        if (!$this->Post_model->exists($id)) {
            // Mengirim response error jika post tidak ditemukan.
            return $this->_response(false, 'Post not found', null, 404);
        }

        // Ambil input request dengan dukungan JSON dan form-data dari Postman.
        $input = $this->_request_input();
        // Menyiapkan array kosong untuk data yang akan diupdate.
        $data = array();
        // Mengulang field yang boleh diupdate.
        foreach (array('title', 'author', 'article') as $field) {
            // Mengecek apakah field tersebut dikirim pada input.
            if (isset($input[$field])) {
                // Memasukkan nilai field ke array update.
                $data[$field] = $input[$field];
            }
        }

        // Mengecek apakah tidak ada data yang dikirim untuk diupdate.
        if (empty($data)) {
            // Mengirim response error jika request tidak membawa data update.
            return $this->_response(false, 'No data to update', null, 400);
        }

        // Menjalankan update data melalui model.
        $this->Post_model->update($id, $data);
        // Mengirim response sukses beserta data terbaru.
        return $this->_response(true, 'Post updated successfully', $this->Post_model->get_by_id($id));
    }

    // Endpoint untuk menghapus data post berdasarkan ID.
    public function delete($id)
    {
        // Mengatur response menjadi JSON.
        $this->_json();
        // Mengecek token JWT sebelum menghapus data.
        if (!$this->_auth_payload()) return;

        // Mengecek apakah post dengan ID tersebut ada.

```

```

        if (!$this->Post_model->exists($id)) {
            // Mengirim response error jika post tidak ditemukan.
            return $this->_response(false, 'Post not found', null, 404);
        }

        // Menghapus data post dari database melalui model.
        $this->Post_model->delete($id);
        // Mengirim response sukses setelah data dihapus.
        return $this->_response(true, 'Post deleted successfully');
    }

    // Method private untuk mengambil dan memverifikasi token JWT.
    private function _auth_payload()
    {
        // Mengambil token dari header Authorization request.
        $token = $this->jwt->get_token_from_request();
        // Memverifikasi token dan mengambil payload-nya.
        $payload = $this->jwt->verify($token);
        // Mengecek apakah token tidak valid.
        if (!$payload) {
            // Mengirim response unauthorized jika token salah atau kosong.
            $this->_response(false, 'Unauthorized or invalid token', null, 401);
            // Mengembalikan false sebagai tanda autentikasi gagal.
            return false;
        }
        // Mengembalikan payload jika token valid.
        return $payload;
    }

    // Method private untuk mengatur response menjadi JSON.
    private function _json()
    {
        // Bersihkan buffer hanya jika memang ada buffer aktif agar tidak memicu
notice.
        if (ob_get_level() > 0) {
            ob_clean();
        }

        // Gunakan output class CI supaya header JSON lebih konsisten.
        $this->output->set_content_type('application/json', 'utf-8');
    }

    // Method private untuk mengambil input request dari JSON, form-data, atau
input stream.
    private function _request_input()
    {
        // Ambil data mentah request untuk mendukung JSON dan x-www-form-
urlencoded.
        $raw_input = trim(file_get_contents('php://input'));

        // Prioritaskan body JSON jika valid.
        if ($raw_input !== '') {
            $decoded = json_decode($raw_input, true);
            if (json_last_error() === JSON_ERROR_NONE && is_array($decoded)) {

```

```

        return $decoded;
    }
}

// POST form-data atau x-www-form-urlencoded.
$post_input = $this->input->post(NULL, true);
if (is_array($post_input) && !empty($post_input)) {
    return $post_input;
}

// PUT/PATCH/DELETE yang dibaca dari input stream.
$stream_input = $this->input->input_stream(NULL, true);
if (is_array($stream_input) && !empty($stream_input)) {
    return $stream_input;
}

// Fallback terakhir jika Postman mengirim body query-string style.
if ($raw_input !== '') {
    parse_str($raw_input, $parsed_input);
    if (is_array($parsed_input) && !empty($parsed_input)) {
        return $parsed_input;
    }
}

// Jika tidak ada input yang valid, kembalikan array kosong.
return array();
}

// Method private untuk mengirim response JSON standar.
private function _response($status, $message, $data = null, $code = 200)
{
    // Mengatur HTTP status code response.
    http_response_code($code);
    // Membuat struktur dasar response API.
    $response = array('status' => $status, 'message' => $message);
    // Mengecek apakah ada data tambahan yang perlu dikirim.
    if ($data !== null) {
        // Menambahkan data ke response.
        $response['data'] = $data;
    }
    // Mencetak response dalam format JSON.
    echo json_encode($response, JSON_UNESCAPED_SLASHES |
JSON_UNESCAPED_UNICODE);
    // Menghentikan eksekusi agar tidak ada output tambahan.
    exit;
}
}

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

/*
| -----
| URI ROUTING

```

This file lets you re-map URI requests to specific controller functions.

Typically there is a one-to-one relationship between a URL string and its corresponding controller class/method. The segments in a URL normally follow this pattern:

example.com/class/method/id/

In some instances, however, you may want to remap this relationship so that a different class/function is called than the one corresponding to the URL.

Please see the user guide for complete details:

http://codeigniter.com/user_guide/general/routing.html

RESERVED ROUTES

There are three reserved routes:

`$route['default_controller'] = 'welcome';`

This route indicates which controller class should be loaded if the URI contains no data. In the above example, the "welcome" class would be loaded.

`$route['404_override'] = 'errors/page_missing';`

This route will tell the Router which controller/method to use if those provided in the URL cannot be matched to a valid route.

`$route['translate_uri_dashes'] = FALSE;`

This is not exactly a route, but allows you to automatically route controller and method names that contain dashes. '-' isn't a valid class or method name character, so it requires translation. When you set this option to TRUE, it will replace ALL dashes in the controller and method URI segments.

Examples: my-controller/index -> my_controller/index
my-controller/my-method -> my_controller/my_method

*/

`$route['default_controller'] = 'Crudjs/Crudjs';`

`$route['404_override'] = '';`

`$route['translate_uri_dashes'] = FALSE;`

`$route['login'] = 'auth';`

`$route['register'] = 'auth/register';`

`$route['logout'] = 'auth/logout';`

`$route['dashboard'] = 'dashboard';`

`$route['peserta'] = 'peserta';`

```

$route['register_API'] = 'Register_API/register_API';
$route['peserta/simpan'] = 'peserta/save';
$route['peserta/hapus/(:num)'] = 'peserta/delete/$1';

// HMVC AJAX CRUD routes
$route['crudjs'] = 'Crudjs/Crudjs';
$route['crudjs/debug'] = 'Crudjs/Crudjs/debug';
$route['crudjs/test'] = 'Crudjs/Crudjs/test';
$route['crudjs/get_all'] = 'Crudjs/Crudjs/get_all';
$route['crudjs/get_record/(:num)'] = 'Crudjs/Crudjs/get_record/$1';
$route['crudjs/store'] = 'Crudjs/Crudjs/store';
$route['crudjs/update/(:num)'] = 'Crudjs/Crudjs/update/$1';
$route['crudjs/delete/(:num)'] = 'Crudjs/Crudjs/delete/$1';

// API Routes
$route['api/auth/register'] = 'Api/Auth/register'; // Endpoint API untuk
mendaftarkan user baru melalui controller Api/Auth method register.
$route['api/auth/login'] = 'Api/Auth/login'; // Endpoint API untuk proses login
user melalui controller Api/Auth method login.
$route['api/auth/logout'] = 'Api/Auth/logout'; // Endpoint API untuk proses logout
user melalui controller Api/Auth method logout.
$route['api/auth/me'] = 'Api/Auth/me'; // Endpoint API untuk mengambil data user
yang sedang login melalui controller Api/Auth method me.
$route['api/post/(:num)'] = 'Api/Post/handle/$1'; // Endpoint API untuk mengelola
satu data post berdasarkan ID angka yang dikirim pada URL.
$route['api/post'] = 'Api/Post/handle'; // Endpoint API untuk mengelola data post
tanpa ID, misalnya menampilkan semua data atau menambah post baru.

```

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
/*
```

```
| -----
| DATABASE CONNECTIVITY SETTINGS
| -----
```

```
| This file will contain the settings needed to access your database.
```

```
| For complete instructions please consult the 'Database Connection'
| page of the User Guide.
```

```
| -----
| EXPLANATION OF VARIABLES
| -----
```

```
|
| ['dsn']          The full DSN string describe a connection to the database.
| ['hostname']     The hostname of your database server.
| ['username']     The username used to connect to the database
| ['password']     The password used to connect to the database
| ['database']     The name of the database you want to connect to
| ['dbdriver']     The database driver. e.g.: mysqli.

```

```
|         Currently supported:
```

```
|             cubrid, ibase, mssql, mysql, mysqli, oci8,
|             odbc, pdo, postgre, sqlite, sqlite3, sqlsrv

```

```
| ['dbprefix']    You can add an optional prefix, which will be added

```

```

|         to the table name when using the Query Builder class
|     ['pconnect'] TRUE/FALSE - Whether to use a persistent connection
|     ['db_debug'] TRUE/FALSE - Whether database errors should be displayed.
|     ['cache_on'] TRUE/FALSE - Enables/disables query caching
|     ['cachedir'] The path to the folder where cache files should be stored
|     ['char_set'] The character set used in communicating with the database
|     ['dbcollat'] The character collation used in communicating with the database
|                     NOTE: For MySQL and MySQLi databases, this setting is only used
|                     as a backup if your server is running PHP < 5.2.3 or MySQL < 5.0.7
|                     (and in table creation queries made with DB Forge).
|                     There is an incompatibility in PHP with mysql_real_escape_string()
which
|
|         can make your site vulnerable to SQL injection if you are using a
|         multi-byte character set and are running versions lower than
these.
|
|         Sites using Latin-1 or UTF-8 database character set and collation
are unaffected.
|     ['swap_pre'] A default table prefix that should be swapped with the dbprefix
|     ['encrypt']  Whether or not to use an encrypted connection.
|
|         'mysql' (deprecated), 'sqlsrv' and 'pdo/sqlsrv' drivers accept
TRUE/FALSE
|         'mysqli' and 'pdo/mysqli' drivers accept an array with the following
options:
|
|             'ssl_key'      - Path to the private key file
|             'ssl_cert'    - Path to the public key certificate file
|             'ssl_ca'      - Path to the certificate authority file
|             'ssl_cacpath' - Path to a directory containing trusted CA
certificats in PEM format
|             'ssl_cipher'  - List of *allowed* ciphers to be used for the
encryption, separated by colons (':')
|             'ssl_verify'  - TRUE/FALSE; Whether verify the server certificate or
not ('mysqli' only)
|
|     ['compress'] Whether or not to use client compression (MySQL only)
|     ['stricton'] TRUE/FALSE - forces 'Strict Mode' connections
|                     - good for ensuring strict SQL while developing
|     ['ssl_options'] Used to set various SSL options that can be used when making
SSL connections.
|     ['failover'] array - A array with 0 or more data for connections if the main
should fail.
|     ['save_queries'] TRUE/FALSE - Whether to "save" all executed queries.
|                     NOTE: Disabling this will also effectively disable both
|                     $this->db->last_query() and profiling of DB queries.
|                     When you run a query, with this setting set to TRUE (default),
|                     CodeIgniter will store the SQL statement for debugging purposes.
|                     However, this may cause high memory usage, especially if you run
|                     a lot of SQL queries ... disable this to avoid that problem.
|
| The $active_group variable lets you choose which connection group to
| make active. By default there is only one group (the 'default' group).
|
| The $query_builder variables lets you determine whether or not to load

```



```

| the query builder class.
*/
$active_group = 'default';
$query_builder = TRUE;

$db['default'] = array(
    'dsn' => '',
    'hostname' => 'localhost',
    'username' => 'root',
    'password' => '',
    'database' => 'webpro',
    'dbdriver' => 'mysqli',
    'dbprefix' => '',
    'pconnect' => FALSE,
    'db_debug' => (ENVIRONMENT !== 'production'),
    'cache_on' => FALSE,
    'cachedir' => '',
    'char_set' => 'utf8',
    'dbcollat' => 'utf8_general_ci',
    'swap_pre' => '',
    'encrypt' => FALSE,
    'compress' => FALSE,
    'stricton' => FALSE,
    'failover' => array(),
    'save_queries' => TRUE
);

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Jwt {

    private $secret_key = 'your_secret_key_change_this_in_production';
    private $algorithm = 'HS256';
    private $expiration = 604800; // 7 days in seconds

    public function __construct()
    {
        $this->secret_key = config_item('jwt_secret_key') ?: $this->secret_key;
        $this->expiration = config_item('jwt_expiration') ?: $this->expiration;
    }

    public function create($payload = array())
    {
        $header = array(
            'typ' => 'JWT',
            'alg' => $this->algorithm
        );

        $payload['iat'] = time();
        $payload['exp'] = time() + $this->expiration;

        $header_encoded = $this->base64_url_encode(json_encode($header));
        $payload_encoded = $this->base64_url_encode(json_encode($payload));
    }
}

```

```

        $signature = hash_hmac(
            'sha256',
            $header_encoded . '.' . $payload_encoded,
            $this->secret_key,
            true
        );
        $signature_encoded = $this->base64_url_encode($signature);

        return $header_encoded . '.' . $payload_encoded . '.' . $signature_encoded;
    }

    public function verify($token = '')
    {
        if (empty($token)) {
            return false;
        }

        $parts = explode('.', $token);
        if (count($parts) !== 3) {
            return false;
        }

        list($header_encoded, $payload_encoded, $signature_encoded) = $parts;

        $signature = hash_hmac(
            'sha256',
            $header_encoded . '.' . $payload_encoded,
            $this->secret_key,
            true
        );
        $signature_computed = $this->base64_url_encode($signature);

        if ($signature_computed !== $signature_encoded) {
            return false;
        }

        $payload = json_decode($this->base64_url_decode($payload_encoded));

        if (isset($payload->exp) && $payload->exp < time()) {
            return false;
        }

        return $payload;
    }

    public function get_token_from_request()
    {
        $auth_header = null;

        if (function_exists('apache_request_headers')) {
            $headers = apache_request_headers();
            $auth_header = isset($headers['Authorization']) ?
$headers['Authorization'] : null;

```

```

    }

    if (!$auth_header && isset($_SERVER['HTTP_AUTHORIZATION'])) {
        $auth_header = $_SERVER['HTTP_AUTHORIZATION'];
    }

    if (!$auth_header && isset($_SERVER['REDIRECT_HTTP_AUTHORIZATION'])) {
        $auth_header = $_SERVER['REDIRECT_HTTP_AUTHORIZATION'];
    }

    if (!$auth_header) {
        return false;
    }

    if (preg_match('/Bearer\s+(\S+)/', $auth_header, $matches)) {
        return $matches[1];
    }

    return false;
}

private function base64_url_encode($input)
{
    return str_replace(array('+', '/', '='), array('-', '_', ''),
base64_encode($input));
}

private function base64_url_decode($input)
{
    $remainder = strlen($input) % 4;
    if ($remainder) {
        $input .= str_repeat('=', 4 - $remainder);
    }

    return base64_decode(str_replace(array('-', '_'), array('+', '/'),
$input));
}
}

<?php
// Mencegah file model diakses langsung dari browser.
defined('BASEPATH') OR exit('No direct script access allowed');

// Model untuk mengelola data user pada API authentication.
class User_model extends CI_Model {

    // Nama tabel database yang digunakan untuk menyimpan data user.
    private $table = 'users';

    // Constructor dijalankan otomatis saat model dipanggil.
    public function __construct()
    {
        // Memanggil constructor bawaan CI_Model.
        parent::__construct();
    }
}

```

```

        // Memuat koneksi database CodeIgniter.
        $this->load->database();
    }

    // Menambahkan user baru ke database.
    public function create($data)
    {
        // Mengisi waktu data dibuat.
        $data['created_at'] = date('Y-m-d H:i:s');
        // Mengisi waktu data terakhir diubah.
        $data['updated_at'] = date('Y-m-d H:i:s');
        // Menjalankan query INSERT ke tabel users.
        $this->db->insert($this->table, $data);
        // Mengembalikan ID user yang baru dibuat.
        return $this->db->insert_id();
    }

    // Mengambil data user berdasarkan ID.
    public function get_by_id($id)
    {
        // Menjalankan query SELECT * FROM users WHERE id = $id dan mengambil satu
        baris.
        return $this->db->get_where($this->table, array('id' => $id))->row();
    }

    // Mengambil data user berdasarkan email.
    public function get_by_email($email)
    {
        // Mengubah email menjadi huruf kecil agar pencarian konsisten.
        return $this->db->get_where($this->table, array('email' =>
        strtolower($email)))->row();
    }

    // Mengecek apakah email sudah terdaftar di database.
    public function email_exists($email)
    {
        // Mengembalikan TRUE jika jumlah data dengan email tersebut lebih dari 0.
        return $this->db->get_where($this->table, array('email' =>
        strtolower($email)))->num_rows() > 0;
    }
}

```

```
<?php
```

```

// Mencegah file model diakses langsung dari browser.
defined('BASEPATH') OR exit('No direct script access allowed');

```

```
// Model untuk mengelola data post pada API.
```

```
class Post_model extends CI_Model {
```

```

    // Nama tabel database yang digunakan untuk menyimpan data post.
    private $table = 'posts';

```

```

    // Constructor dijalankan otomatis saat model dipanggil.

```

```
    public function __construct()
```

```

{
    // Memanggil constructor bawaan CI_Model.
    parent::__construct();
    // Memuat koneksi database CodeIgniter.
    $this->load->database();
}

// Mengambil semua data post dari database.
public function get_all()
{
    // Mengurutkan data dari ID terbaru lalu mengembalikan hasil sebagai array
    object.
    return $this->db->order_by('id', 'DESC')->get($this->table)->result();
}

// Mengambil satu data post berdasarkan ID.
public function get_by_id($id)
{
    // Menjalankan query SELECT * FROM posts WHERE id = $id dan mengambil satu
    baris.
    return $this->db->get_where($this->table, array('id' => $id))->row();
}

// Menambahkan post baru ke database.
public function create($data)
{
    // Mengisi waktu data dibuat.
    $data['created_at'] = date('Y-m-d H:i:s');
    // Mengisi waktu data terakhir diubah.
    $data['updated_at'] = date('Y-m-d H:i:s');
    // Menjalankan query INSERT ke tabel posts.
    $this->db->insert($this->table, $data);
    // Mengembalikan ID post yang baru dibuat.
    return $this->db->insert_id();
}

// Mengubah data post berdasarkan ID.
public function update($id, $data)
{
    // Memperbarui waktu update setiap kali data diubah.
    $data['updated_at'] = date('Y-m-d H:i:s');
    // Menentukan data mana yang akan diupdate berdasarkan ID.
    $this->db->where('id', $id);
    // Menjalankan query UPDATE dan mengembalikan status berhasil/gagal.
    return $this->db->update($this->table, $data);
}

// Menghapus data post berdasarkan ID.
public function delete($id)
{
    // Menentukan data mana yang akan dihapus berdasarkan ID.
    $this->db->where('id', $id);
    // Menjalankan query DELETE dan mengembalikan status berhasil/gagal.
    return $this->db->delete($this->table);
}

```

```

    }

    // Mengecek apakah data post dengan ID tertentu ada.
    public function exists($id)
    {
        // Mengembalikan TRUE jika jumlah data dengan ID tersebut lebih dari 0.
        return $this->db->get_where($this->table, array('id' => $id))->num_rows() >
0;
    }
}

CREATE DATABASE IF NOT EXISTS `webpro`;
USE `webpro`;

CREATE TABLE IF NOT EXISTS `posts` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `title` varchar(255) NOT NULL,
  `author` varchar(255) NOT NULL,
  `article` text NOT NULL,
  `image` varchar(255) DEFAULT NULL,
  `created_at` datetime DEFAULT NULL,
  `updated_at` datetime DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS `users` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL,
  `created_at` datetime DEFAULT NULL,
  `updated_at` datetime DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

<?php
// Memanggil partial head untuk kebutuhan tampilan halaman register.
$this->load->view('partials/head', array('title' => $title));
?>
<main class="auth-body">
  <section class="auth-container">
    <div class="auth-copy">
      <p class="auth-badge">Registrasi</p>
      <h1>Buat akun baru</h1>
      <p class="auth-subtitle">Akun ini dipakai untuk masuk ke dashboard
utama aplikasi.</p>
    </div>

    <?php
    // Menampilkan pesan validasi atau status proses register.
    $this->load->view('partials/alerts');
    ?>

```

```

<?php
// Form register mengirim data akun baru ke controller Auth.
echo form_open('register', array('class' => 'auth-form'));
?>

    <label>Nama Lengkap</label>
    <input type="text" name="username" value="<?php echo
set_value('username'); ?>" placeholder="Masukkan nama lengkap" required>

    <label>Email</label>
    <input type="email" name="email" value="<?php echo set_value('email');
?>" placeholder="Masukkan email" required>

    <label>Password</label>
    <input type="password" name="password" placeholder="Minimal 5 karakter"
required>

    <button type="submit" class="btn-primary">Daftar</button>
    <?php echo form_close(); ?>

    <p class="auth-switch">Sudah punya akun? <a href="<?php echo
site_url('login'); ?>">Login</a></p>
</section>
</main>
<?php
// Memanggil partial footer untuk menutup struktur halaman.
$this->load->view('partials/footer');
?>

<?php
echo $data;
?>

```

```

<?php
// Memanggil partial head untuk elemen <head> dan asset utama.
$this->load->view('partials/head', array('title' => $title));
?>
<main class="auth-body">
    <section class="auth-container">
        <div class="auth-copy">
            <p class="auth-badge">CodeIgniter 3</p>
            <h1>Masuk ke InkluSkill</h1>
            <p class="auth-subtitle">Dashboard hanya bisa diakses setelah login
berhasil, sesuai instruksi modul.</p>
        </div>

        <?php
        // Menampilkan notifikasi error atau sukses jika ada.
        $this->load->view('partials/alerts');
        ?>

        <?php
        // Form login mengirim data ke route login.
        echo form_open('login', array('class' => 'auth-form'));
        ?>

```

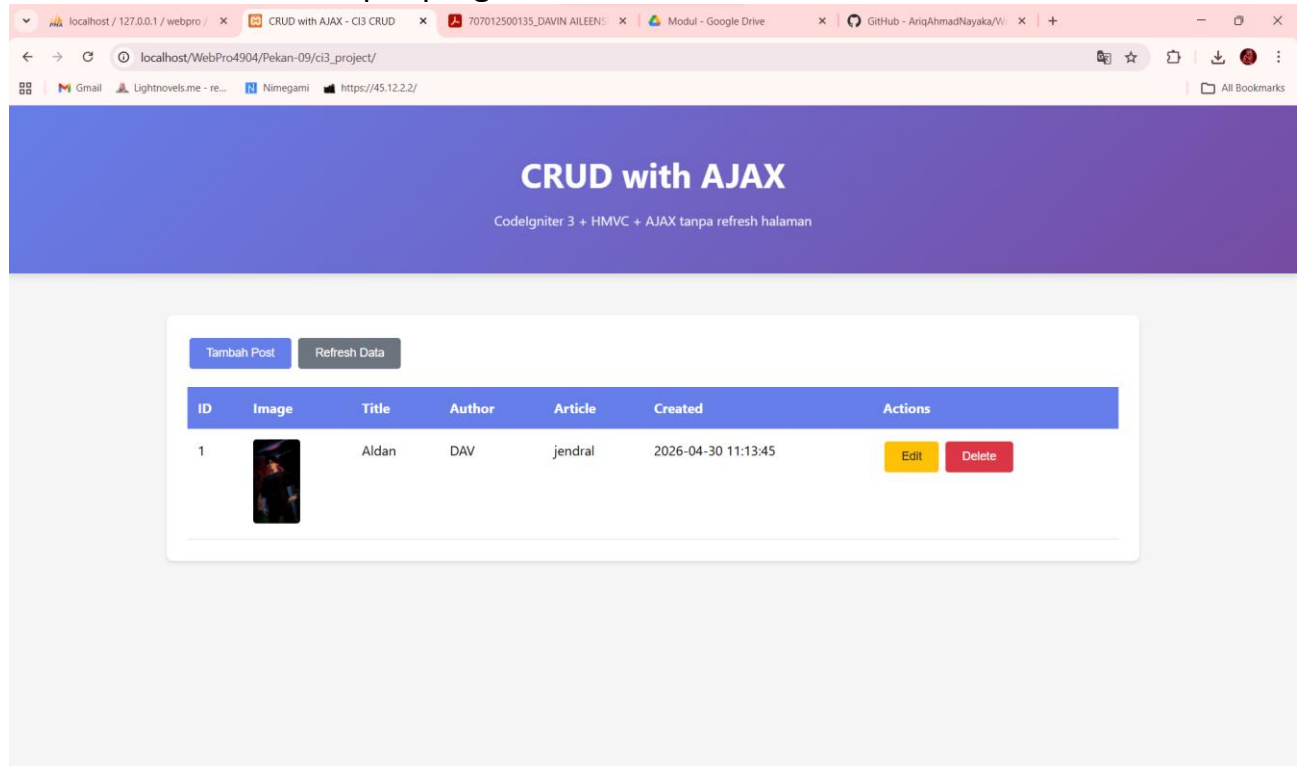
```
<label>Email</label>
<input type="email" name="email" value="<?php echo set_value('email');
?>" placeholder="Masukkan email" required>

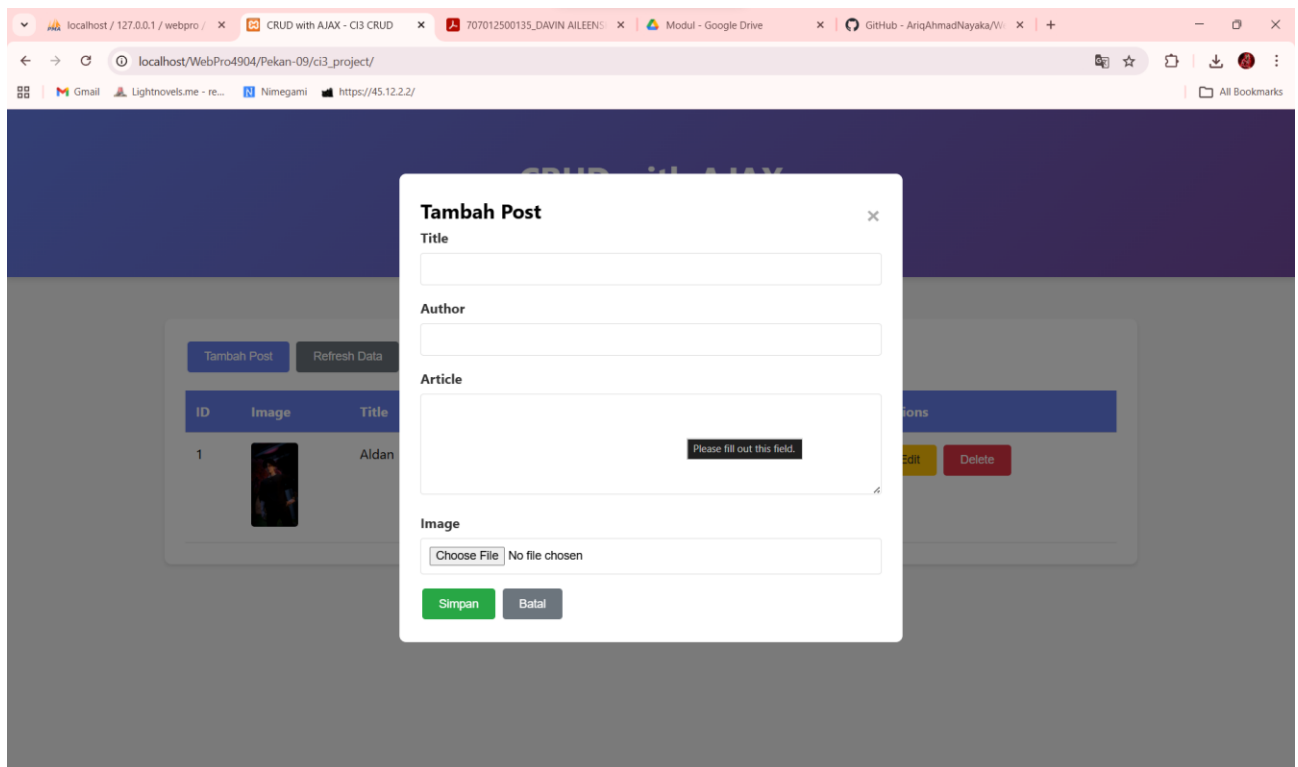
<label>Password</label>
<input type="password" name="password" placeholder="Masukkan password"
required>

<button type="submit" class="btn-primary">Login</button>
<?php echo form_close(); ?>

<p class="auth-switch">Belum punya akun? <a href="<?php echo
site_url('register'); ?>">Daftar</a></p>
</section>
</main>
<?php
// Memanggil partial footer untuk script JS penutup.
$this->load->view('partials/footer');
?>
```

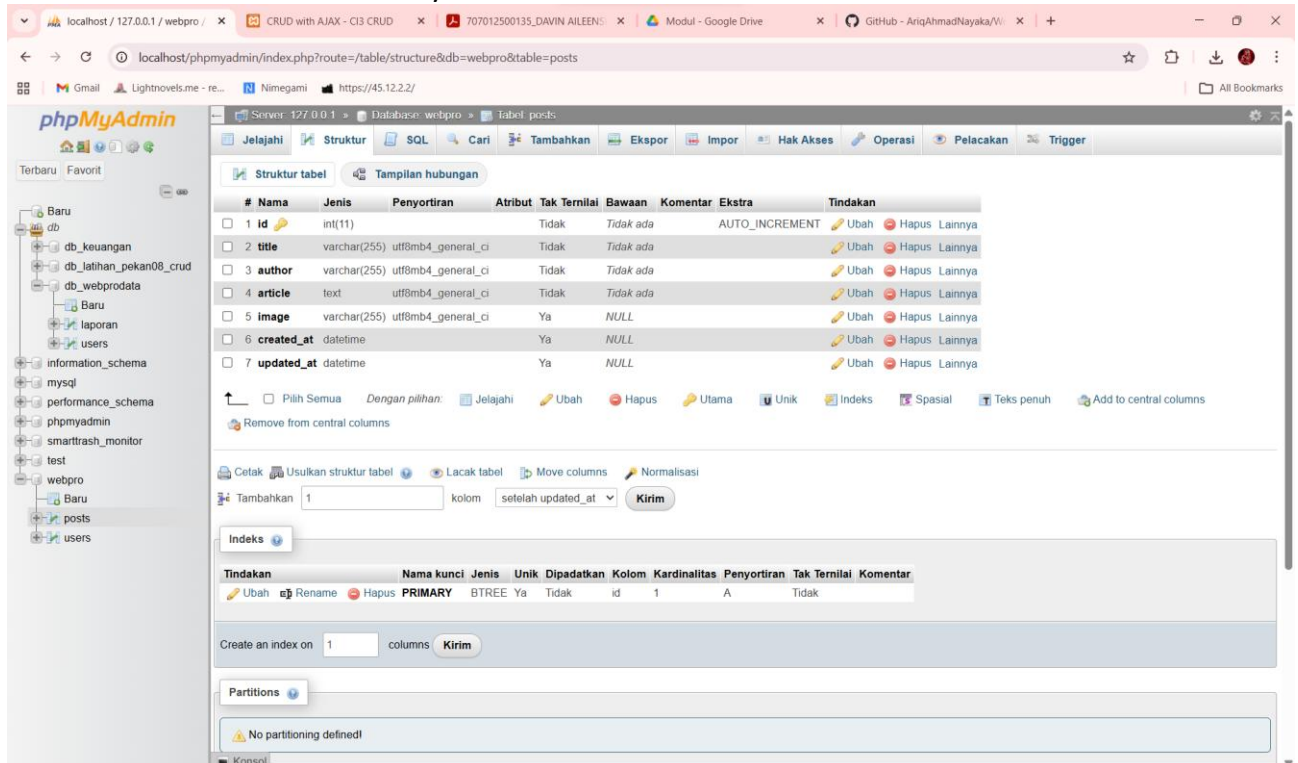

ini adalah hasil output program





Ini adalah program edit CRUD nya

Ini adalaah struktur database nya



localhost / 127.0.0.1 / webpro / x

CRUD with AJAX - C13 CRUD x 707012500135_DAVIN AILEEN S x Modul - Google Drive x GitHub - AriqAhmadNayaka/W x

localhost/phpmyadmin/index.php?route=/table/structure&db=webpro&table=users

Server: 127.0.0.1 > Database: webpro > Tabel: users

Jelajahi Struktur SQL Cari Tambahkan Ekspor Impor Hak Akses Operasi Pelacakan Trigger

Struktur tabel Tampilan hubungan

#	Nama	Jenis	Penyortiran	Atribut	Tak Ternilai	Bawaan	Komentar	Ekstra	Tindakan
1	id	int(11)		Tidak	Tidak ada			AUTO_INCREMENT	Ubah Hapus Lainnya
2	name	varchar(255)	utf8mb4_general_ci	Tidak	Tidak ada				Ubah Hapus Lainnya
3	email	varchar(255)	utf8mb4_general_ci	Tidak	Tidak ada				Ubah Hapus Lainnya
4	password	varchar(255)	utf8mb4_general_ci	Tidak	Tidak ada				Ubah Hapus Lainnya
5	created_at	datetime		Ya	NULL				Ubah Hapus Lainnya
6	updated_at	datetime		Ya	NULL				Ubah Hapus Lainnya

Pilih Semua Dengan pilihan: Jelajahi Ubah Hapus Utama Unik Indeks Spasial Teks penuh Add to central columns

Remove from central columns

Cetak Usulkan struktur tabel Lacak tabel Move columns Normalisasi

Tambahkan 1 kolom setelah updated_at Kirim

Indeks

Tindakan	Nama kunci	Jenis	Unik	Dipadatkan	Kolom	Kardinalitas	Penyortiran	Tak Ternilai	Komentar
Ubah Rename Hapus	PRIMARY	BTREE	Ya	Tidak	id	2	A	Tidak	
Ubah Rename Hapus	email	BTREE	Ya	Tidak	email	2	A	Tidak	

Create an index on 1 columns Kirim

Partitions

No partitioning defined!

Konsol

Ini adalah isi database nya

localhost / 127.0.0.1 / webpro / x | CRUD with AJAX - C13 CRUD | 707012500135_DAVIN AILENS | Modul - Google Drive | GitHub - ArianAhmadNayaka/W... | +

localhost/phpmyadmin/index.php?route=/sql&pos=0&db=webpro&table=posts

phpMyAdmin

Server: 127.0.0.1 > Database: webpro > Tabel: posts

Menampilkan baris 0 - 0 (total 1, Pencarian dilakukan dalam 0,0001 detik)

SELECT * FROM `posts`

Profil [Edit dikotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini

Extra options

	id	title	author	article	image	created_at	updated_at				
☐	Ubah	☐	Salin	☐	Hapus	1	Aldan DAV	jendral	posts/1777522425_AldanM_2.jpg	2026-04-30 11:13:45	2026-04-30 11:13:45

Pilih Semua Dengan pilihan: Ubah Salin Hapus Ekspor

Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini

Operasi hasil kueri

Cetak Salin ke clipboard Ekspor Tampilkan bagan Buat tampilan

Markahi kueri SQL ini

Judul: Izinkan semua pengguna untuk mengakses markah ini

Markahi kueri SQL ini

Konsol

localhost / 127.0.0.1 / webpro / x | CRUD with AJAX - C13 CRUD | 707012500135_DAVIN AILENS | Modul - Google Drive | GitHub - ArianAhmadNayaka/W... | +

localhost/phpmyadmin/index.php?route=/sql&pos=0&db=webpro&table=users

phpMyAdmin

Server: 127.0.0.1 > Database: webpro > Tabel: users

Menampilkan baris 0 - 1 (total 2, Pencarian dilakukan dalam 0,0002 detik)

SELECT * FROM `users`

Profil [Edit dikotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini | Sort by key: Tidak ada

Extra options

	id	name	email	password	created_at	updated_at					
☐	Ubah	☐	Salin	☐	Hapus	1	Davin	davin@test.com	\$2y\$10\$XgXhZ/BZrdENDIQFL68HZFbLchuOVio4QeZTk sk...	2026-04-30 11:11:58	2026-04-30 11:11:58
☐	Ubah	☐	Salin	☐	Hapus	2	Davin Suka Kak Arian	davinmiaw@gmail.com	\$2y\$10\$B00iACyN7MJSTaJ0V46eegFcNu0mbeBnN38jUFVuU...	2026-04-30 11:12:42	2026-04-30 11:12:42

Pilih Semua Dengan pilihan: Ubah Salin Hapus Ekspor

Tampilkan semua | Jumlah baris: 25 | Saring baris: Cari di tabel ini | Sort by key: Tidak ada

Operasi hasil kueri

Cetak Salin ke clipboard Ekspor Tampilkan bagan Buat tampilan

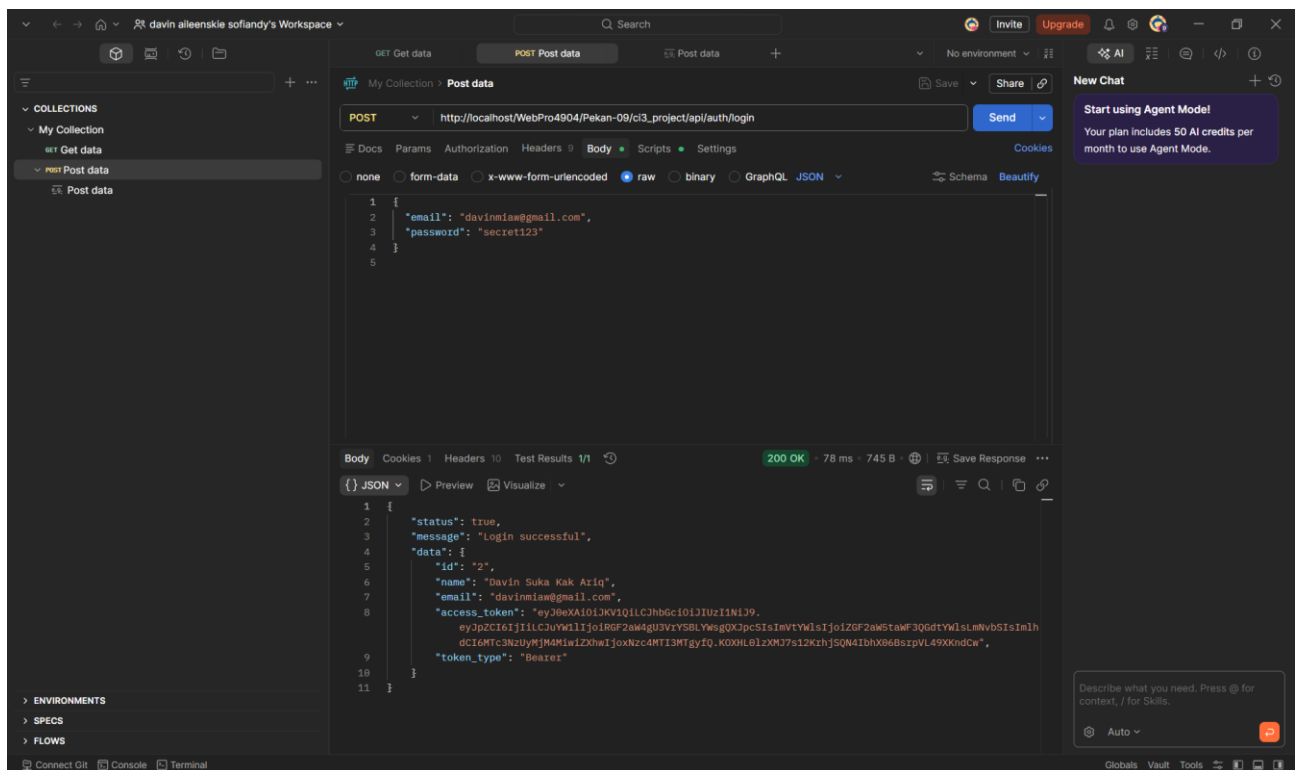
Markahi kueri SQL ini

Judul: Izinkan semua pengguna untuk mengakses markah ini

Markahi kueri SQL ini

Konsol

POSTMAN



E. Analisis dan Pembahasan

Analisis

Pada praktikum ini, saya mempelajari penerapan REST API pada CodeIgniter 3 dengan pola HMVC. Implementasi dibagi menjadi dua bagian utama, yaitu autentikasi user dan CRUD data post.

Bagian autentikasi menggunakan token JWT. Setelah user berhasil login atau register, sistem membuat token yang nantinya dipakai untuk mengakses endpoint yang membutuhkan otorisasi. Dengan cara ini, data post tidak bisa diakses sembarang tanpa token.

Masalah utama yang saya temukan saat pengujian di Postman adalah respons API tidak bersih karena muncul notice PHP sebelum JSON ditampilkan. Hal ini terjadi karena fungsi `ob_clean()` dipanggil saat tidak ada output buffer aktif. Akibatnya, Postman menerima output campuran HTML error dan JSON.

Selain itu, pembacaan input request sebelumnya terlalu bergantung pada raw JSON, sehingga request dari Postman bisa gagal jika format body berbeda. Karena itu, saya memperbaiki proses pembacaan input agar endpoint lebih fleksibel menerima berbagai jenis body request.

Dari hasil perbaikan tersebut, API menjadi lebih stabil dan lebih mudah diuji. Response yang dihasilkan menjadi konsisten dalam format JSON dan endpoint dapat menerima request dari Postman dengan lebih baik.

Penggunaan FormData penting karena form juga mendukung upload gambar. Pada proses tampil data, fungsi loadRecords() akan mengambil data dari endpoint get_all. Jika data berhasil diterima, JavaScript membuat baris tabel secara otomatis berdasarkan response JSON. Jika tidak ada data, tabel akan menampilkan pesan bahwa data masih kosong.

Pada proses edit data, tombol Edit memanggil fungsi editPost(id). Fungsi ini mengambil data berdasarkan ID melalui endpoint get_record/{id}, lalu mengisi form modal dengan data lama. Setelah itu, pengguna bisa mengubah isi form dan menyimpannya kembali.

Pada proses hapus data, fungsi deletePost(id) terlebih dahulu menampilkan konfirmasi. Jika pengguna menyetujui, AJAX mengirim request ke endpoint delete/{id}. Setelah data berhasil dihapus, tabel dimuat ulang agar tampilan sesuai dengan isi database terbaru.

Secara keseluruhan, fitur ini menunjukkan penerapan konsep CRUD, AJAX, JSON response, MVC/HMVC, validasi form, dan upload gambar dalam satu module. Pembagian file antara controller, model, dan view membuat kode lebih terstruktur. Controller menangani logika proses, model menangani database, sedangkan view dan JavaScript menangani tampilan serta interaksi pengguna.

F. Kesimpulan

Praktikum Pekan 09 berhasil menghasilkan REST API sederhana berbasis CodeIgniter 3 yang mencakup fitur autentikasi dan CRUD post. API dapat diuji melalui Postman menggunakan endpoint register, login, logout, me, dan post.

Dari praktikum ini, saya memahami cara membuat routing API, controller HMVC, koneksi database, penggunaan JWT untuk autentikasi, serta proses pengujian API di Postman. Saya juga memahami bahwa penanganan response JSON dan pembacaan input request sangat penting agar API berjalan dengan baik.

LINK REPOSITORI GITHUB DAVIN

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/DavinAileenskieSofiandy-707012500135>